

Diseño de la interfaz REST

API REST de videojuegos con Node.js, Express y MongoDB

1. Objetivo del documento

Este documento explica las decisiones de diseño de la interfaz REST del proyecto. La especificación formal completa se mantiene en docs/openapi.yaml; por tanto, aquí se resumen las reglas de diseño, los recursos, los formatos de mensaje y ejemplos representativos alineados con la implementación actual.

La API permite consultar y gestionar videojuegos, desarrolladores y reviews almacenados en MongoDB. Además, expone información de países en XML, generada a partir de datos integrados desde Wikidata y asociada a videojuegos y desarrolladores.

Documento	Responsabilidad
Diseno_Interfaz_REST.docx	Criterios de diseño REST y ejemplos de mensajes representativos.
openapi.yaml	Contrato formal completo: rutas, parámetros, cuerpos, schemas y códigos de respuesta.
modelo-datos.md	Colecciones MongoDB, campos, relaciones y decisiones del modelo de datos.
countries.xsd	Schema asociado a las respuestas XML de países.

2. Principios de diseño REST

- Recursos en plural: /games, /developers, /reviews y /countries.
- El método HTTP expresa la operación: GET consulta, POST crea, PUT reemplaza/actualiza, PATCH modifica parcialmente y DELETE elimina.
- Los identificadores públicos viajan en la ruta: /games/{id}, /developers/{id}, /reviews/{id}. No se expone el _id interno de MongoDB.
- La búsqueda, paginación, filtros y ordenación se expresan mediante query params.
- Las relaciones se exponen como subrecursos cuando dependen de un videojuego: /games/{id}/reviews y /games/{id}/country.
- Los recursos modificables trabajan con JSON. Los países se devuelven en XML y tienen un XSD asociado.

3. Recursos de la API

Recurso	Colección	Formato principal	Operaciones	Relación principal
/games	videogames	JSON	GET, POST, PUT, DELETE	Recurso central; su id conecta con reviews y countries.
/developers	developers	JSON	GET, POST, PUT, DELETE	Cada desarrollador contiene juegos asociados en games[id].
/reviews	reviews	JSON	GET, POST, PATCH, DELETE	Cada review apunta a un videojuego mediante gameId.
/countries	countries	XML	GET	Relaciona gameId, gameName, developer y country.

4. Formatos de mensaje

JSON

Los recursos /games, /developers y /reviews devuelven JSON. Las operaciones de creación y modificación también reciben application/json. Las respuestas evitan exponer campos internos de MongoDB como _id.

```
Content-Type: application/json
Accept: application/json
```

XML y XSD

El recurso /countries y el subrecurso /games/{id}/country devuelven application/xml. La estructura XML tiene el schema asociado docs/countries.xsd, con raíz countries y una o varias entradas country.

```
<?xml version="1.0" encoding="UTF-8"?>
<countries>
  <country>
    <gameId>3328</gameId>
    <gameName>The Witcher 3: Wild Hunt</gameName>
    <developer>CD Projekt RED</developer>
    <country>Poland</country>
  </country>
</countries>
```

5. Contrato resumido por recurso

/games

Representa videojuegos importados inicialmente desde RAWG y cargados en MongoDB. Es la colección de mayor volumen y permite búsquedas con filtros, ordenación y paginación.

Método y ruta	Descripción	Parámetros principales
GET /games	Lista videojuegos.	id, search, platform, genre, store, minRating, page, limit, sort
GET /games/{id}	Obtiene un videojuego concreto.	id en ruta
GET /games/{id}/reviews	Obtiene reviews del videojuego.	id en ruta
GET /games/{id}/country	Obtiene países asociados en XML.	id en ruta
GET /games/{id}/enriched	Agrega videojuego, países y reviews.	id en ruta
POST /games	Crea un videojuego.	JSON con id, name, platforms, genres y stores
PUT /games/{id}	Actualiza un videojuego.	JSON con campos permitidos
DELETE /games/{id}	Elimina un videojuego si no tiene dependencias.	id en ruta

/developers

Representa estudios o desarrolladores. Se puede consultar por nombre, ordenar, paginar y filtrar por videojuegos asociados mediante gameId.

Método y ruta	Descripción	Parámetros principales
GET /developers	Lista desarrolladores.	search, gameId, page, limit, sort
GET /developers/{id}	Obtiene un desarrollador concreto.	id en ruta

POST /developers	Crea un desarrollador.	JSON con id, name, slug y games
PUT /developers/{id}	Actualiza un desarrollador.	JSON con campos permitidos
DELETE /developers/{id}	Elimina un desarrollador.	id en ruta

/reviews

Representa opiniones propias del proyecto. Se separa de /games para poder crear, modificar y eliminar reviews sin tocar la información importada desde RAWG.

Método y ruta	Descripción	Parámetros principales
GET /reviews	Lista reviews.	id, rating, page, limit
GET /reviews/game/{gameId}	Obtiene reviews asociadas a un videojuego.	gameId en ruta
POST /reviews	Crea una review.	JSON con id, gameId, gameName, rating y comment
PATCH /reviews/{id}	Modifica parcialmente una review.	JSON con gameId, gameName, user, rating o comment
DELETE /reviews/{id}	Elimina una review.	id en ruta

/countries

Representa la relación entre videojuego, desarrollador y país. Es un recurso de consulta en XML, pensado para cubrir el requisito XML y enriquecer la información de videojuegos.

Método y ruta	Descripción	Parámetros principales
GET /countries	Lista países en XML.	country, developer, gameName
GET /games/{id}/country	Lista países asociados a un videojuego en XML.	id en ruta

6. Ejemplos de mensajes

Ejemplo 1: búsqueda paginada de videojuegos

```
GET /games?search=witcher&platform=pc&genre=rpg&page=1&limit=1&sort=-rating HTTP/1.1
Host: localhost:3001
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{
  "total": 1,
  "videogames_length": 1,
  "videogames": [
    {
      "id": 3328,
      "slug": "the-witcher-3-wild-hunt",
      "name": "The Witcher 3: Wild Hunt",
      "released": "2015-05-18",
      "rating": 4.7,
      "platforms": ["PC", "PlayStation 4"],
      "genres": ["RPG", "Adventure"],
      "stores": ["Steam", "GOG"]
    }
  ]
}
```

```
}  
]  
}
```

Ejemplo 2: creación de un videojuego

```
POST /games HTTP/1.1  
Host: localhost:3001  
Content-Type: application/json  
Accept: application/json  
  
{  
  "id": 999999,  
  "slug": "hollow-echoes",  
  "name": "Hollow Echoes",  
  "released": "2026-01-01",  
  "rating": 4.2,  
  "platforms": ["PC"],  
  "genres": ["Adventure"],  
  "stores": ["Steam"],  
  "esrb_rating": "Everyone"  
}  
  
HTTP/1.1 201 Created  
Content-Type: application/json  
  
{  
  "message": "Videojuego creado correctamente",  
  "id": 999999,  
  "game": {  
    "id": 999999,  
    "slug": "hollow-echoes",  
    "name": "Hollow Echoes",  
    "released": "2026-01-01",  
    "rating": 4.2,  
    "platforms": ["PC"],  
    "genres": ["Adventure"],  
    "stores": ["Steam"],  
    "esrb_rating": "Everyone"  
  }  
}
```

Ejemplo 3: reviews de un videojuego

```
GET /reviews/game/3328 HTTP/1.1  
Host: localhost:3001  
Accept: application/json  
  
HTTP/1.1 200 OK  
Content-Type: application/json  
  
{  
  "reviews_length": 1,  
  "reviews": [  
    {  
      "id": 1,  
      "gameId": 3328,  
      "gameName": "The Witcher 3: Wild Hunt",  
      "user": "player_01",  
      "rating": 5,  
      "comment": "Muy buen juego",  
      "createdAt": "2025-01-01T00:00:00.000Z"  
    }  
  ]  
}
```

Ejemplo 4: actualización parcial de una review

```
PATCH /reviews/1 HTTP/1.1
Host: localhost:3001
Content-Type: application/json
Accept: application/json

{
  "rating": 4,
  "comment": "Sigue siendo excelente, aunque el inicio puede hacerse lento."
}

HTTP/1.1 200 OK
Content-Type: application/json

{
  "message": "Review actualizada correctamente",
  "review": {
    "id": 1,
    "gameId": 3328,
    "gameName": "The Witcher 3: Wild Hunt",
    "user": "player_01",
    "rating": 4,
    "comment": "Sigue siendo excelente, aunque el inicio puede hacerse lento."
  }
}
```

Ejemplo 5: respuesta XML de países

```
GET /games/3328/country HTTP/1.1
Host: localhost:3001
Accept: application/xml

HTTP/1.1 200 OK
Content-Type: application/xml

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<countries>
  <country>
    <gameId>3328</gameId>
    <gameName>The Witcher 3: Wild Hunt</gameName>
    <developer>CD Projekt RED</developer>
    <country>Poland</country>
  </country>
</countries>
```

7. Códigos de respuesta y validaciones

Código	Uso	Ejemplo de situación
200	Consulta o actualización correcta.	GET /games/3328, PATCH /reviews/1
201	Recurso creado.	POST /games, POST /developers, POST /reviews
400	Parámetro o cuerpo inválido.	limit no numérico, rating fuera de rango, campos no permitidos
404	Recurso inexistente.	Videojuego, desarrollador, review o país no encontrado
409	Conflicto de integridad.	ID duplicado o borrado de videojuego con recursos relacionados
500	Error interno no controlado.	Fallo inesperado de servidor o base de datos

Validaciones principales

- Los IDs deben ser enteros positivos.

- page y limit deben ser enteros positivos.
- rating de videojuegos debe estar entre 0 y 5; rating de reviews, entre 1 y 5.
- POST /games exige id, name, platforms, genres y stores.
- POST /developers exige id, name, slug y games; cada juego asociado debe existir en videogames.
- POST /reviews exige id, gameld, gameName, rating y comment; gameName debe coincidir con el videojuego indicado.
- No se permite exponer ni modificar directamente el _id interno de MongoDB.

8. Relación con el modelo de datos

La interfaz REST refleja el modelo de datos documentado en docs/modelo-datos.md. El identificador videogames.id conecta con reviews.gameld y countries.gameld. Los desarrolladores guardan sus videojuegos asociados en developers.games[].id, lo que permite filtrar /developers mediante el query param gameld.

La ruta /games/{id}/enriched no representa una colección nueva. Es una vista de lectura que agrupa el videojuego base, sus países asociados y sus reviews para facilitar el consumo desde el cliente.

9. Resumen de cumplimiento

Requisito del proyecto	Dónde queda cubierto
API REST en Node.js con MongoDB	Implementación en api/ con Express y driver de MongoDB.
Mensajes JSON	Recursos /games, /developers y /reviews.
Mensaje XML con schema asociado	/countries, /games/{id}/country y docs/countries.xsd.
Al menos tres recursos relacionados	/games, /developers, /reviews y /countries relacionados por gameld y games[].id.
CRUD sobre la base de datos	CRUD en /games y /developers; POST/PATCH/DELETE en /reviews; /countries es consulta XML.
Dataset y carga automática	api/datasets/ y scripts npm run seed definidos en api/package.json.
Paginación y filtrado en colección grande	GET /games con page, limit y filtros sobre videogames.
Descripción formal del servicio	docs/openapi.yaml.

10. Referencia final

Para validar el contrato completo se debe tomar docs/openapi.yaml como fuente formal. Este documento evita duplicar todos los schemas y rutas en detalle para reducir contradicciones, y se centra en justificar el diseño de la interfaz y mostrar ejemplos representativos.